



Embedded System Lab Manual

LAB EXPERIMENT NO. 5

Serial UART Base STM32 Programming with STM32F103C8T6 Blue Pill C++

Submitted to :

Mr Yasir

Submitted by:

Ahsan Basharat

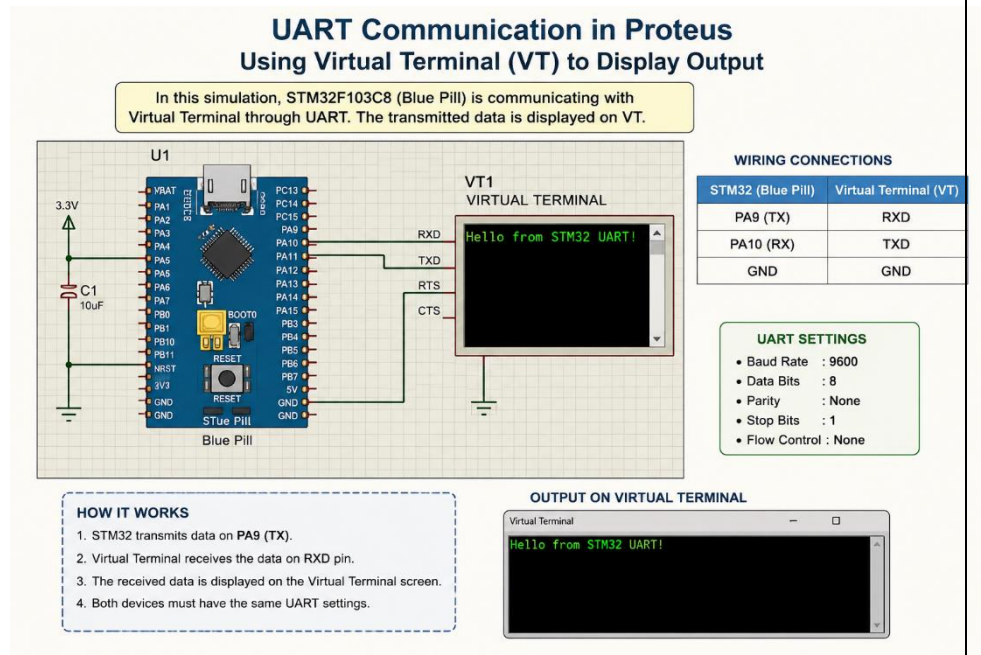
Date of Submission:

5/07/2027

Introduction

In this lab, we studied UART (Universal Asynchronous Receiver and Transmitter), which is one of the most commonly used serial communication protocols in embedded systems. UART enables communication between two devices without requiring a shared clock signal, making it an asynchronous communication method. It uses two communication lines: TX (Transmit) for sending data and RX (Receive) for receiving data. In this experiment, we configured the UART peripheral on the STM32 microcontroller using STM32CubeMX and the HAL library. We then transmitted data from the STM32 to a serial terminal and also implemented an echo program, where the microcontroller receives data and immediately sends the same data back to the sender. This lab helped us understand the basic working principle of UART communication, including data transmission, data reception, baud rate configuration, and serial communication between devices.

In this lab, we used Proteus simulation to verify whether the data was transmitted correctly through UART communication. A Virtual Terminal was connected to the STM32 UART pins to monitor the transmitted data. When the program was executed, the Virtual Terminal displayed the transmitted message, confirming that the UART communication was working correctly. Proteus provided a convenient way to test and validate the UART functionality before implementing it on actual hardware

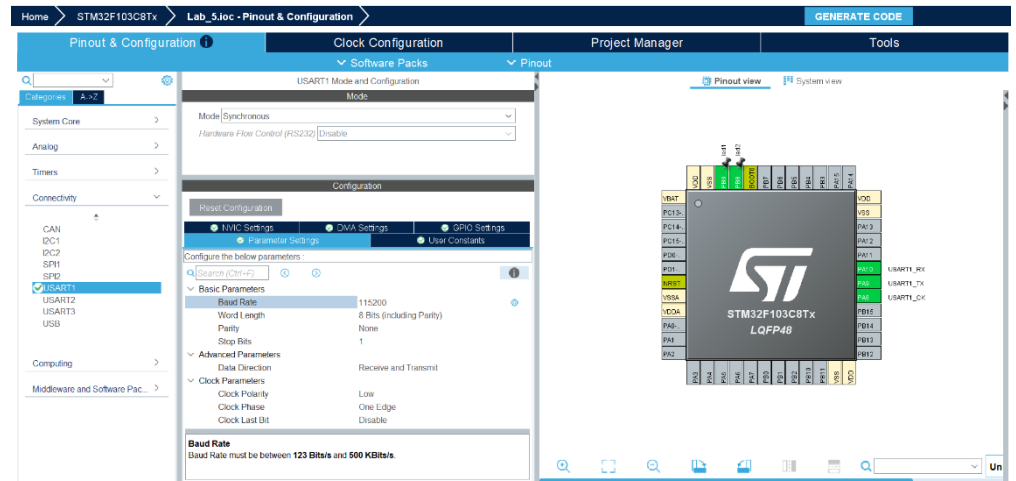


STM32CudeMX

In this lab, we followed the same procedure as in the previous lab to create a new STM32 project in STM32CubeMX. After creating the project, we enabled UART (USART1)

and configured it in Asynchronous mode. We then set the basic communication parameters according to the STM32 board, including the baud rate, word length,

parity, and stop bits. For this experiment, we used the STM32F103C8T6 (Blue Pill) development board, which provides three USART peripherals (USART1, USART2, and USART3). In this lab, we used USART1 for serial communication. After completing the UART configuration, we generated the project code and opened it in STM32CubeIDE for programming and testing.



STM32CudeIDE

After configuring the UART peripheral in STM32CubeMX, we created three source files: uart_hal.h, uart_hal.cpp, and main.cpp. The purpose of these files was to implement UART communication using the STM32 HAL (Hardware Abstraction Layer) library. In uart_hal.h, we created a class named UART to encapsulate all UART-related operations. The class contains a constructor, a destructor, and member functions for transmitting and receiving data. The constructor accepts a pointer to the USART_HandleTypeDef structure and stores it so that the UART peripheral can be accessed throughout the program.

```
#ifndef UART_HAL_H
#define UART_HAL_H

#include "main.h"
#include "usart.h"

class UART
{
public:
    UART(USART_HandleTypeDef *huart);
    ~UART();

    HAL_StatusTypeDef write(uint8_t *data,
                           uint16_t size,
                           uint32_t timeout = HAL_MAX_DELAY);

    HAL_StatusTypeDef read(uint8_t *data,
                           uint16_t size,
                           uint32_t timeout = HAL_MAX_DELAY);

private:
    USART_HandleTypeDef *Instance;
};

#endif
```

HAL_UART Function

```
HAL_StatusTypeDef HAL_UART_Transmit(USART_HandleTypeDef *husart, const uint8_t *pTxData, uint16_t Size, uint32_t Timeout);
HAL_StatusTypeDef HAL_UART_Receive(USART_HandleTypeDef *husart, uint8_t *pRxData, uint16_t Size, uint32_t Timeout);
HAL_StatusTypeDef HAL_UART_TransmitReceive(USART_HandleTypeDef *husart, const uint8_t *pTxData, uint8_t *pRxData,
```

HAL_StatusTypeDef HAL_UART_Transmitter(USART_HandleTypeDef *husart, uint8_t *pTxData, uint16_t Size, uint32_t Timeout);

- HAL_UART_Transmit() does not return the transmitted data. Instead, it returns a status indicating whether the transmission was successful or if an error occurred.
- USART_HandleTypeDef *husart specifies which USART peripheral (such as USART1 or USART2) will be used to transmit the data.
- const uint8_t *pTxData is a pointer to the data that you want to send. It stores the memory address of the data to be transmitted.
- uint16_t Size specifies the number of bytes to transmit. For example, the string "Hello" contains 5 bytes, so the size would be 5.
- uint32_t Timeout specifies the maximum time the function will wait for the transmission to complete if the USART is busy. The timeout value is given in milliseconds (ms). For example, a timeout value of 1000 means the function will wait for up to 1 second before returning HAL_TIMEOUT.

Read and Write Function on UART

The write() function is used to send (transmit) data through the UART/USART peripheral.

- The function returns a HAL_StatusTypeDef value (HAL_OK, HAL_ERROR, HAL_BUSY, or HAL_TIMEOUT) to indicate whether the transmission was successful.

The read() function is used to receive data from the UART/USART peripheral.

- The function returns a HAL_StatusTypeDef value to indicate whether the data was received successfully or if an error, busy state, or timeout occurred.

```
HAL_StatusTypeDef UART::write(uint8_t *data,
                               uint16_t size,
                               uint32_t timeout)
{
    return HAL_UART_Transmit(
        Instance,
        data,
        size,
        timeout);
}

HAL_StatusTypeDef UART::read(uint8_t *data,
                              uint16_t size,
                              uint32_t timeout)
{
    return HAL_UART_Receive(
        Instance,
        data,
        size,
        timeout);
}
```

Main.cpp

First we Initialize the `MX_USART1_Init()` for USART1 with the baud rate, word length, parity, stop bits, and other settings configured in STM32CubeMX.

Creates an object named `uart1` of the UART class and passes the address of `husart1`. This object will use USART1 for communication.

```
uint8_t msg[] = "Hello World";
```

Creates an array of type `uint8_t` containing the string "Hello World".

```
uart1.write(msg, sizeof(msg) - 1);
```

1. Calls the `write()` function.
2. Sends the message "Hello World" through USART1.
3. `sizeof(msg)` returns 12 because the string contains:

- 11 characters (Hello World)
- 1 null character ('\0')

4. `sizeof(msg) - 1` equals 11, so only the actual message is transmitted.

5. `HAL_Delay(1000);`

Pauses the program for 1000 milliseconds (1 second) before sending the message again.

```
void MX_USART1_Init(void)
{
    /* USER CODE BEGIN USART1_Init 0 */
    /* USER CODE END USART1_Init 0 */
    /* USER CODE BEGIN USART1_Init 1 */
    /* USER CODE END USART1_Init 1 */
    husart1.Instance = USART1;
    husart1.Init.BaudRate = 115200;
    husart1.Init.WordLength = USART_WORDLENGTH_8B;
    husart1.Init.StopBits = USART_STOPBITS_1;
    husart1.Init.Parity = USART_PARITY_NONE;
    husart1.Init.Mode = USART_MODE_TX_RX;
    husart1.Init.CLKPolarity = USART_POLARITY_LOW;
    husart1.Init.CLKPhase = USART_PHASE_1EDGE;
    husart1.Init.CLKLastBit = USART_LASTBIT_DISABLE;
}
```

```
#include "main.h"
#include "usart.h"
#include "uart_hal.h"

void SystemClock_Config(void);

int main(void)
{
    HAL_Init();

    SystemClock_Config();

    // MX_GPIO_Init();
    MX_USART1_Init();

    UART uart1(&husart1);

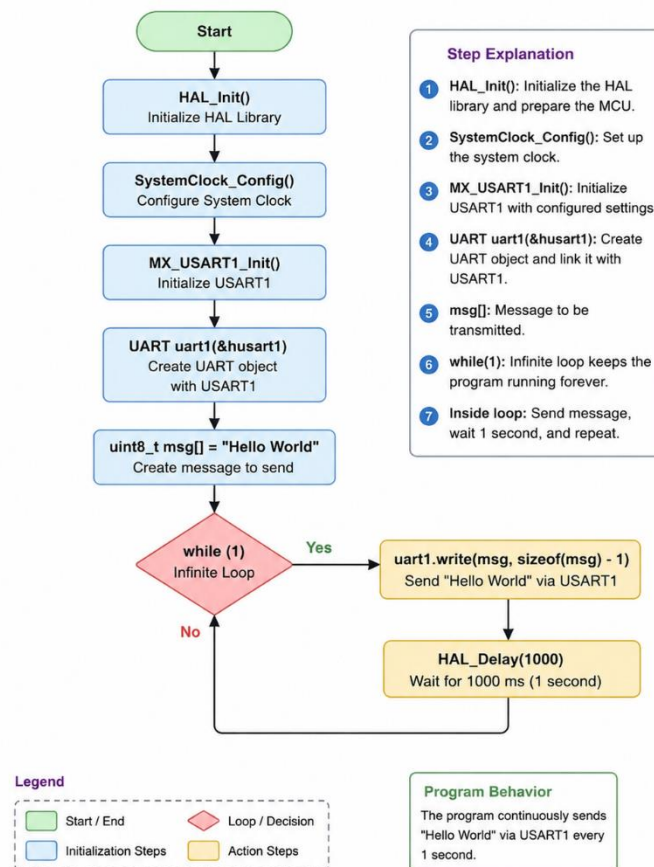
    uint8_t msg[] = "Hello World";

    while (1)
    {
        uart1.write(msg, sizeof(msg) - 1);

        HAL_Delay(1000);
    }
}
```

UART Communication Flow Graph:

Flow Graph of main() Function

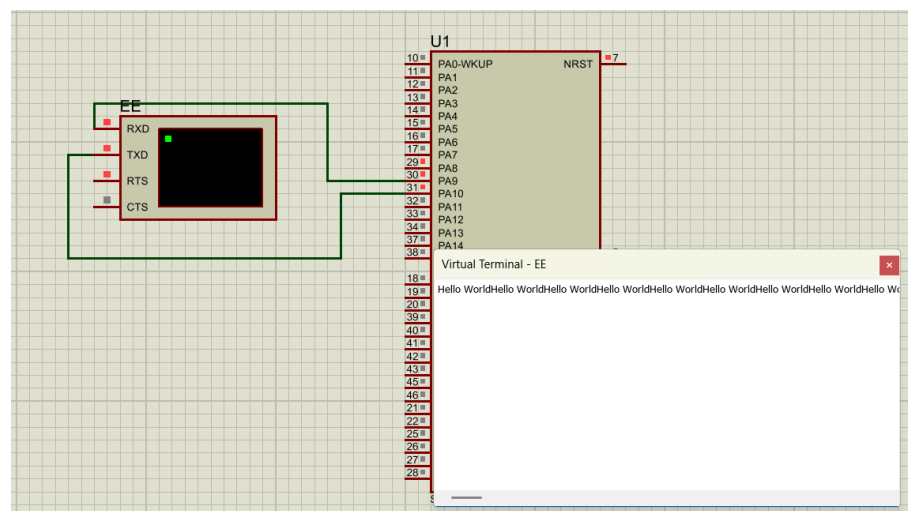


Proteus Output:

In UART communication, the TX (Transmit) pin of one device is always connected to the RX (Receive) pin of the other device. This allows one device to send data while the other receives it.

In this project:

- STM32 PA9 (USART1_TX) is connected to the RXD pin of the Virtual Terminal. This connection allows the STM32 to send



the "Hello World" message to the Virtual Terminal.

- STM32 PA10 (USART1_RX) is connected to the TXD pin of the Virtual Terminal. This connection is used when the Virtual Terminal sends data back to the STM32.

In this program, the STM32 only sends data because the write() function is used. The message "Hello World" is transmitted through PA9 (USART1_TX) and received by the RXD pin of the Virtual Terminal, where it is displayed on the screen. The PA10 (USART1_RX) to TXD connection is not used in this program. It will be used later when the read() function is implemented, allowing the STM32 to receive data sent from the Virtual Terminal.